

12장 사용자 정의 자료형

12.1 구조체

12.2 공용체

12.3 열거체



학습목표

- 사용자 정의 자료형의 기본 개념과 사용상의 장점을 설명할 수 있다.
- 구조체를 정의하고 사용할 수 있다.
- 구조체 변수와 구조체 포인터의 멤버에 접근하는 방법을 설명할 수 있다.
- 구조체 변수를 함수의 인수로 전달하는 프로그램을 작성할 수 있다.
- 공용체를 정의하고 사용할 수 있다.
- 구조체와 공용체의 멤버들의 메모리 구조의 차이를 설명할 수 있다.
- 열거체를 정의하고 사용할 수 있다.

사용자 정의 자료형

- 사용자 정의 자료형

- 여러 다른 자료형으로 선언된 변수들을 묶어서 하나의 의미 있는 자료의 단위가 되도록 만든 새로운 자료형
- 새로운 형태의 자료형에 포함될 각 멤버들의 자료형, 변수 이름 등을 알려주어 컴파일러가 기억공간의 크기, 자료형의 특징을 알 수 있도록 선언하여 주어야 한다.
- 종류 : 구조체, 공용체, 열거체
- 사용자 정의 자료형은 붕어빵이라는 변수를 만들어 내는 붕어빵 틀과 같다.



구조체

- 구조체(structure)란

- (예) 학생의 간단한 인적 사항을 프로그램으로 작성하는 경우 학생의 학번, 이름, 나이 정보가 필요하다.

이러한 정보를 표현하는 변수를 선언하면

- 학번을 넣을 변수 : `int id;`
 - 이름을 넣을 변수 : `char *name;`
 - 나이를 넣을 변수 : `int age;`
- 이런 경우 여러 다른 자료형의 변수들을 하나로 묶어서 새로운 자료형처럼 사용할 수 있는 것이 구조체이다.
 - 배열은 동일한 자료형의 변수들의 모임
 - 구조체는 다른 자료형의 데이터를 하나로 묶은 것

구조체

- 구조체의 정의

```
struct 구조체_이름 {  
    자료형 변수_이름; // 멤버 변수 1  
    자료형 변수_이름; // 멤버 변수 2  
    ....  
}; // 마지막에 반드시 ';'을 붙여야 한다.
```

- 구조체 변수의 선언

- C 언어에서 구조체 변수를 선언하려면 구조체 이름(구조체 자료형) 앞에 반드시 struct를 붙여야 한다.

```
struct 구조체_이름 변수_이름;
```

구조체

- (예) 구조체의 정의 및 구조체 변수 선언의 예

```
struct Student {           // 구조체의 정의
    int id;
    char *name;
    int age;
```

```
};
```

```
struct Student Kim;        // 구조체 변수의 선언
```

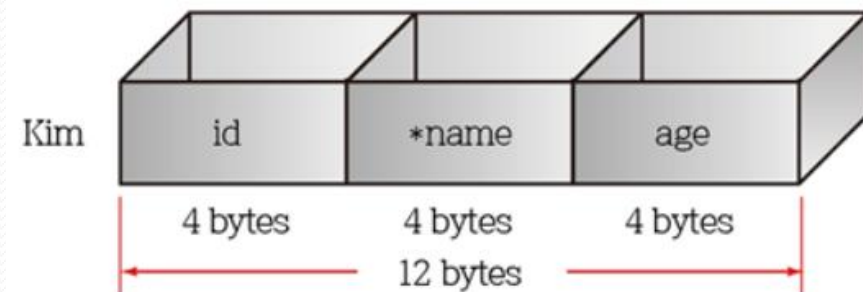
```
struct Student {           // 구조체의 정의와 동시에 변수 선언
    int id;
    char *name;
    int age;
} Kim;
```

```
struct {                   // 구조체의 자료형 이름 없이 정의와 동시에 변수 선언
    int id;
    char *name;
    int age;
} Lee;
```

구조체

● 구조체 변수의 메모리 구조

- 앞의 예에서 struct는 구조체를 정의하기 위한 예약어이다.
- 구조체 이름 Student는 새로운 자료형의 이름이다.
- Student 자료형을 구성하는 id, name, age 변수를 구조체 Student의 필드(field) 혹은 멤버 변수라고 한다.
- 앞의 예에서 구조체 Student의 변수 Kim은 구조체의 멤버 변수 크기만큼 즉, 정수형 변수(4 byte)와 포인터 변수(4 byte) 그리고 정수형 변수(4 byte)의 크기만큼 메모리를 연속적으로 확보한다. 즉, 구조체 변수의 전체 크기는 총 12 byte의 메모리로 할당된다.



구조체

● 예제 12.1

- 복소수(Complex)를 나타내는 구조체를 정의하고, 변수 comp1, comp2를 선언하시오.
멤버 변수로 실수(real, 배정도 부동소수점 수),
허수(image, 배정도 부동소수점 수)를 갖는다.

- (풀이)

```
struct Complex {  
    double real;  
    double image;  
};  
struct Complex comp1, comp2;
```

혹은

```
struct Complex {  
    double real;  
    double image;  
} comp1, comp2;
```


구조체

- 구조체 변수의 멤버에 접근하기
 - 구조체 변수의 멤버에 접근하려면
'구조체_변수_이름 . 멤버 변수'
와 같이 사용자 정의 자료형 변수의 **멤버 참조 연산자(.)**를 사용하여 멤버에 접근할 수 있다.
 - 여기서 '.'은 멤버 참조 연산자이다.

구조체

- 구조체 변수에 데이터를 초기화하기

```
구조체_변수_이름 . 멤버_변수 = 데이터;
```

(예)

```
struct Student Kim;           // 구조체 변수 선언
Kim.id = 20161234;             // 멤버 변수의 초기화
Kim.name = "김선달";           // 멤버 변수의 초기화
Kim.age = 22;                  // 멤버 변수의 초기화
struct Student Hong = {20161512, "홍길동", 21};
// 구조체 변수의 선언과 동시에 초기화하는 경우,
// 멤버들의 초기값을 주는 순서는 구조체 정의에서의
// 순서와 같아야 한다.
```

구조체

- 예제 12.2 (ex12_2.c)

```
#include <stdio.h>
```

```
struct Student {  
    int id;  
    char *name;  
    int age;  
};
```

```
void main()  
{
```

```
    struct Student Hong = {20161512, (char *)"홍길동", 21};  
    struct Student Kim;  
    Kim.id = 20161234;  
    Kim.name = (char *)"김선달";  
    Kim.age = 22;
```

```
    printf("학번 : %d, 이름 : %s, 나이 : %d \n", Hong.id, Hong.name, Hong.age);  
    printf("학번 : %d, 이름 : %s, 나이 : %d \n", Kim.id, Kim.name, Kim.age);
```

```
}
```

```
학번 : 20161512, 이름 : 홍길동, 나이 : 21  
학번 : 20161234, 이름 : 김선달, 나이 : 22  
계속하려면 아무 키나 누르십시오 . . .
```

구조체

- 구조체의 재정의
 - C 언어에서 구조체 변수 선언은 구조체 이름(구조체 자료형) 앞에 반드시 struct를 붙여야 한다.
 - typedef를 이용하면 자료형의 이름을 재정의 할 수 있다.
 - (예) struct Point 라는 자료형 이름 대신에 point 라는 새로운 자료형 이름을 사용하는 예

```
struct Point {                                // 구조체의 정의
    int x;
    int y;
};
```

```
typedef struct Point point;                  // 구조체의 재정의
```

```
typedef struct Point {                        // 구조체의 정의 및 재정의
    int x;
    int y;
} point;
```

```
point p1, p2;                               // 구조체 변수의 선언
```

구조체

- 예제 12.3 (ex12_3.c)

```
#include <stdio.h>
```

```
struct Point {  
    int x;  
    int y;  
};  
typedef struct Point point;
```

```
void main()  
{  
    struct Point A;  
    point B = {5, 8};  
    A.x = 3;  
    A.y = 4;  
  
    printf("A점의 좌표 값: (%d, %d) \n", A.x, A.y);  
    printf("B점의 좌표 값: (%d, %d) \n", B.x, B.y);  
}
```

```
A점의 좌표 값: (3, 4)  
B점의 좌표 값: (5, 8)  
계속하려면 아무 키나 누르십시오 . . .
```

구조체

- 실습문제 12.1 (1)

- 멤버가 책 제목, 저자, 출판 년도로 구성된 Book 이라는 구조체를 만들고, 구조체 변수를 선언한 후 각각의 멤버 변수에 데이터를 초기화하고, 결과를 모니터에 출력하는 프로그램을 작성하시오.

구조체

- 실습문제 12.1 (px12_1.c) (2)

```
#include <stdio.h>
```

```
struct Book {  
    char title[40];  
    char author[40];  
    unsigned int year;  
};
```

```
void main()  
{  
    struct Book book1 = {"염소의 축제", "마리오 바르가스 요사", 2010};  
  
    printf("%s\n", book1.title);  
    printf("%s\n", book1.author);  
    printf("%d\n", book1.year);  
}
```

```
염 소의 축제  
마 리오 바르가스 요사  
2010  
계속하려면 아무 키나 누르십시오 . . .
```

구조체

- 구조체 배열 (1)
 - 구조체 배열의 선언

```
struct 구조체_이름 변수_이름[크기];
```

(예)

```
struct Student {  
    int id;  
    char *name;  
    int age;  
};  
struct Student class_A[3];
```

- 각 구조체 배열 원소의 멤버
class_A[0].id, class_A[0].name, class_A[0].age
class_A[1].id, class_A[1].name, class_A[1].age
class_A[2].id, class_A[2].name, class_A[2].age

구조체

- 구조체 배열 (2)

- 구조체 배열 원소의 초기화

(예)

```
struct Student class_A[3] = { {20161001, "김선달", 22},  
                               {20161002, "임꺽정", 21}, {20161003, "홍길동", 20} };
```

- 각 구조체 배열 원소의 멤버의 값

```
class_A[0].id = 20161001, class_A[0].name = "김선달", class_A[0].age = 22  
class_A[1].id = 20161002, class_A[1].name = "임꺽정", class_A[1].age = 21  
class_A[2].id = 20161003, class_A[2].name = "홍길동", class_A[2].age = 20
```

구조체

- 예제 12.4 (ex12_4.c)

```
#include <stdio.h>
```

```
struct Student {  
    int id;  
    char *name;  
    int age;
```

```
};
```

```
void main()  
{
```

```
    int a;
```

```
    struct Student class_A[3] = { {20161001, "김선달", 22},  
                                    {20161002, "임꺽정", 21}, {20161003, "홍길동", 20} };  
    for (a=0; a<3; a++) {  
        printf("학번: %d, 이름: %s, 나이: %d \n",  
               class_A[a].id, class_A[a].name, class_A[a].age);  
    }  
}
```

```
학번 : 20161001, 이름 : 김선달, 나이 : 22  
학번 : 20161002, 이름 : 임꺽정, 나이 : 21  
학번 : 20161003, 이름 : 홍길동, 나이 : 20  
계속하려면 아무 키나 누르십시오 . . .
```

구조체

- 구조체의 중첩 사용 (1)

- 구조체는 그 멤버 변수로 다른 구조체 변수를 포함할 수 있다.
- (예) 원을 나타내는 구조체 (1)
 - 원을 표현하려면 중심점의 x 좌표, y 좌표와 반지름을 나타내는 r 로 3개의 변수를 가져야 한다.
 - 그러므로 원에 대한 구조체는 다음과 같이 정의할 수 있다.

```
struct Circle {  
    int center_x;  
    int center_y;  
    int radius;  
};
```

구조체

- 구조체의 중첩 사용 (2)

- (예) 원을 나타내는 구조체 (2)

- 하나의 점의 위치를 나타내는 구조체 Point가 정의되어 있다면, 구조체 Point의 변수를 멤버로 사용하여 구조체 Circle을 선언하면 원을 다음과 같이 표현할 수 있다.

```
struct Point {  
    int x;  
    int y;  
};
```

```
struct Circle {  
    struct Point center;  
    int radius;  
};
```

// 구조체를 멤버로 사용한 구조체

```
struct Circle cir_1;  
cir_1.center.x = 4;  
cir_1.center.y = 5;  
cir_1.radius = 10;
```

// 구조체 변수 cir_1의 선언
// 구조체 변수 cir_1의 멤버 참조
// 구조체 변수 cir_1의 멤버 참조
// 구조체 변수 cir_1의 멤버 참조

구조체

- 예제 12.5 (ex12_5.c)

```
#include <stdio.h>

struct Point {
    int x;
    int y;
};

struct Circle {
    struct Point center;
    int radius;
};
```

```
void main()
{
    struct Circle cir_1;

    cir_1.center.x = 4;
    cir_1.center.y = 5;
    cir_1.radius = 10;

    printf("원 cir_1의 중심점의 위치: (%d, %d) \n",
        cir_1.center.x, cir_1.center.y);
    printf("원 cir_1의 반경: %d \n", cir_1.radius);
}
```

```
원 cir_1의 중심점의 위치: (4, 5)
원 cir_1의 반경: 10
계속하려면 아무 키나 누르십시오 . . .
```

구조체

- 실습문제 12.2 (1)

- x 좌표와 y 좌표를 가지는 점을 나타내는 구조체 Point를 정의하고, 두 점을 멤버로 가지는 직선을 나타내는 구조체 Line을 정의하시오. 구조체 Line을 이용하여 직선을 저장하고 그 직선의 길이를 구하는 프로그램을 작성하시오. 길이를 구할 때는 13장에 있는 함수 `sqrt()`를 사용하시오.

구조체

- 실습문제 12.2 (px12_2.c) (2)

```
#include <stdio.h>
#include <math.h>

struct Point {
    int x;
    int y;
};

struct Line {
    struct Point p1;
    struct Point p2;
};

void main()
{
    struct Line ln;
    float lx, ly;
    double length;
```

구조체

- 실습문제 12.2 (px12_2.c) (3)

```
ln.p1.x = 4;  
ln.p1.y = 6;  
ln.p2.x = 14;  
ln.p2.y = 16;
```

```
p1의 좌표 : (4, 6)  
p2의 좌표 : (14, 16)  
ln의 길이 : 14.142136  
계속하려면 아무 키나 누르십시오 . . .
```

```
lx = (float)(ln.p2.x - ln.p1.x);  
ly = (float)(ln.p2.y - ln.p1.y);  
length = sqrt(lx * lx + ly * ly);
```

```
printf("p1의 좌표 : (%d, %d)\n", ln.p1.x, ln.p1.y);  
printf("p2의 좌표 : (%d, %d)\n", ln.p2.x, ln.p2.y);  
printf("ln의 길이 : %lf\n", length);
```

```
}
```


구조체

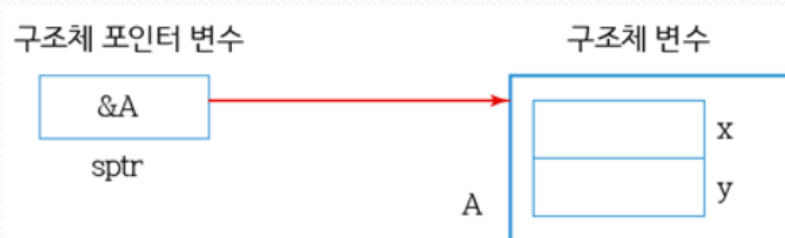
● 구조체 포인터 (1)

- 구조체 포인터 변수는 구조체를 가리키는 포인터 변수이다.

(예)

```
struct Point {           // 2차원 좌표의 점을 표현하는 구조체의 정의
    int x;
    int y;
};

struct Point A;           // struct Point형 구조체의 변수 선언
struct Point *sptr;       // struct Point형 구조체 포인터 변수를 선언
sptr = &A;                // 포인터 변수를 구조체 변수 A의 주소 값으로 초기화
                          // 크기는 sptr가 4 bytes, A가 8 bytes 이다.
```



구조체

- 구조체 포인터 (2)

- 구조체 변수의 멤버 변수에 접근
 - 구조체 변수의 멤버 참조 연산자인 점 연산자(.)를 사용
- 구조체 포인터 변수의 멤버 변수에 접근
 1. 간접 연산자(dereference operator)(*)를 사용
 2. 구조체 포인터 변수를 직접 사용하여 포인터가 가리키는 구조체의 멤버 변수에 접근하려면 사용자 정의 자료형에 대한 **구조체 포인터 변수의 멤버 참조 연산자인 화살표 연산자(->)**를 사용한다.

구조체

- 구조체 포인터 (3)

- (예) 멤버에 접근하는 방법

```
struct Point A;           // A : 구조체 변수  
struct Point *sptr = &A;  // sptr : 구조체 포인터 변수
```

```
A.x = 4;                  // 구조체 변수로 멤버 변수에 접근  
A.y = 5;
```

```
(*sptr).x = 4;            // 구조체 포인터 변수의 간접 연산자를 사용  
(*sptr).y = 5;           // *sptr은 A와 같은 의미이다.
```

```
sptr->x = 4;              // 구조체 포인터 변수로 직접 멤버 변수에  
접근  
sptr->y = 5;
```

구조체

- 예제 12.6 (ex12_6.c) (1)

```
#include <stdio.h>
```

```
struct Point {  
    int x;  
    int y;  
    int z;  
};  
typedef struct Point point;
```

```
void main()  
{  
    point A = {3, 4, 5};  
    point B = {2, 6, 9};  
    point *ptr1, *ptr2;  
  
    ptr1 = &A;  
    ptr2 = &B;
```

구조체

- 예제 12.6 (ex12_6.c) (2)

```
printf("A점의 좌표 : (%d, %d, %d) \n", A.x, A.y, A.z);  
printf("A점의 좌표 : (%d, %d, %d) \n", (*ptr1).x, (*ptr1).y, (*ptr1).z);  
printf("B점의 좌표 : (%d, %d, %d) \n", ptr2->x, ptr2->y, ptr2->z);  
}
```

```
A점의 좌표 : (3, 4, 5)  
A점의 좌표 : (3, 4, 5)  
B점의 좌표 : (2, 6, 9)  
계속하려면 아무 키나 누르십시오 . . .
```

구조체

- 실습문제 12.3 (1)

- 점을 나타내는 구조체 Point와 직선을 나타내는 구조체 Line을 이용하여 직선을 저장하고 그 직선의 길이를 구하는 프로그램을 구조체 포인터 변수를 사용하여 작성하시오.

구조체

- 실습문제 12.3 (px12_3.c) (2)

```
#include <stdio.h>
#include <math.h>

struct Point {
    int x;
    int y;
};

struct Line {
    struct Point p1;
    struct Point p2;
};

void main()
{
    struct Line ln;
    struct Line *pLn = &ln;
    float lx, ly;
    double length;
```

구조체

- 실습문제 12.3 (px12_3.c) (3)

```
lx = (float)(pln->p2.x - pln->p1.x);  
ly = (float)(pln->p2.y - pln->p1.y);  
length = sqrt(lx * lx + ly * ly);
```

```
printf("p1의 좌표 : (%d, %d)\n", pln->p1.x, pln->p1.y);  
printf("p2의 좌표 : (%d, %d)\n", pln->p2.x, pln->p2.y);  
printf("ln의 길이 : %lf\n", length);
```

```
}
```

```
p1의 좌표 : (4, 6)  
p2의 좌표 : (14, 16)  
ln의 길이 : 14.142136  
계속하려면 아무 키나 누르십시오 . . .
```


구조체

- 함수의 인수와 반환 값을 구조체로 전달하기
 - 구조체 변수를 인수로 전달하면 기본적으로 값에 의한 전달이다.
 - 반환 값으로 구조체를 사용하면 여러 개의 데이터를 하나의 구조체 데이터로 만들어 되돌려줄 수 있다.

- (예)

```
struct Student {  
    int id;  
    char name[20];  
};
```

```
void stuPrn(struct Student temp) {  
    printf("학번 : %d, 이름 : %s\n", temp.id, temp.name);  
}
```

```
main()  
{  
    struct Student stu = {1234, "Hong Gildong"};  
    stuPrn(stu);  
}
```

구조체

- 예제 12.7 (ex12_7.c) (1)

```
#include <stdio.h>
```

```
typedef struct Student {
```

```
    int id;
```

```
    char name[10];
```

```
    int age;
```

```
} student;
```

```
void stuPrn(student temp);
```

```
student stuInput();
```

구조체

- 예제 12.7 (ex12_7.c) (2)

```
int main()
{
    student stu1;

    stu1 = stuInput();
    printf("%n 학번 이름 나이 %n");
    printf("===== %n");
    stuPrn(stu1);
    printf("%n");
    return 0;
}

void stuPrn(student temp)
{
    printf("%d %s %d %n", temp.id, temp.name, temp.age);
}
```

구조체

- 예제 12.7 (ex12_7.c) (3)

```
student stuInput()
{
    student temp;

    printf("학번을 입력하세요: ");
    scanf_s("%d", &temp.id);
    printf("이름을 입력하세요: ");
    scanf_s("%s", temp.name, 10);
    printf("나이를 입력하세요: ");
    scanf_s("%d", &temp.age);

    return temp;
}
```

```
학번을 입력하세요: 20161234
이름을 입력하세요: 홍길동
나이를 입력하세요: 20

학번      이름      나이
=====
20161234   홍길동    20

계속하려면 아무 키나 누르십시오 . . .
```

공용체

● 공용체 (1)

- 공용체(union) 구문은 구조체와 같지만 멤버 구조의 의미는 다르다.
- 공용체는 여러 멤버 변수에 해당하는 데이터들이 동일한 메모리 공간을 공통으로 사용
- 동일한 메모리 공간을 여러 가지 형의 변수를 사용하여 기억공간을 공유하고자 할 때 공용체를 사용
- 공용체의 정의

```
union 공용체명 {  
    자료형 변수명;           // 멤버 데이터 1  
    자료형 변수명;           // 멤버 데이터 2  
  
    ....  
};    // 마지막에 반드시 ';'을 붙여야 한다.
```

공용체

- 공용체 (2)

- (예)

```
union value {                                // value 공용체의 정의
    char   c;
    int    i;
    double d;
};
typedef union value number; // value 공용체의 재정의

void main()
{
    union value num1;                        // value 공용체 변수 num1의 선언
    number num2;                             // value 공용체 변수 num2의 선언
    num1.c = 61;                            // char형만 저장
    num1.i = 541494;                        // int형만 저장, char형 값은 없어짐
    num1.d = 3.141592;                     // double형만 저장,
                                           // char형과 int형 값은 없어짐
    .....
}
```

공용체

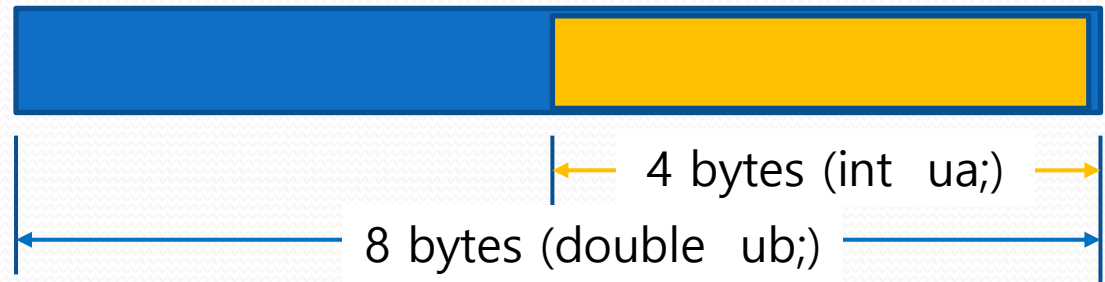
- 구조체와 공용체의 메모리 구조 비교 (1)
 - 예를 들면 char형과 int형과 double형의 데이터가 있을 때,
 - 구조체
 - 이들 모두를 동시에 별도의 메모리 공간에 보관
 - 공용체
 - 동일한 메모리 공간에 세 종류의 데이터 중 한 순간에는 어느 하나만 보관
 - 공용체 변수는 char형이나 int형이나 double형의 데이터를 보관할 수는 있지만 어느 한 시점에는 어느 하나의 데이터만 저장한다.
 - 공용체는 예를 들어 32 bit 데이터를 32 bit 데이터로 사용하기도 하고 16 bit 데이터로 사용하기도 하는 경우에 편리하게 사용할 수 있다.

공용체

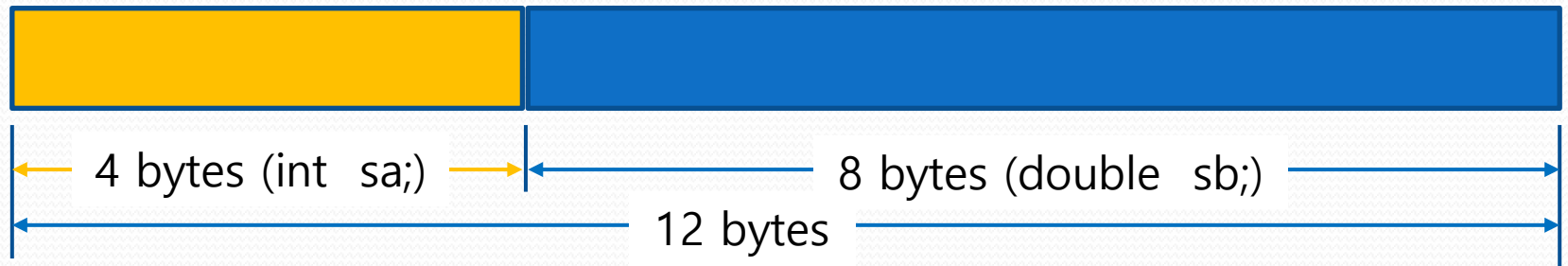
- 구조체와 공용체의 메모리 구조 비교 (2)

(예)

```
union Uni {  
    int ua;  
    double ub;  
};
```



```
struct Str {  
    int sa;  
    double sb;  
};
```



공용체

- 예제 12.8 (ex12_8.c) (1)

```
#include <stdio.h>
```

```
union value {  
    char  c;  
    int   i;  
    double d;  
};
```

```
typedef union value number;
```

```
int main()
```

```
{
```

```
    number num;
```

```
    num.c = 61;
```

```
    printf("num.c = %d 를 보관하는 경우\n", num.c);
```

```
    printf(" num.c = %d, num.i = %d, num.d = %e\n", num.c, num.d, num.d);
```

공용체

- 예제 12.8 (ex12_8.c) (2)

```
num.i = 541494;
printf("num.i = %d 를 보관하는 경우\n", num.i);
printf(" num.c = %d, num.i = %d, num.d = %e \n", num.c, num.d, num.d);

num.d = 3.141592;
printf("num.d = %lf 를 보관하는 경우\n", num.d);
printf(" num.c = %d, num.i = %d, num.d = %e \n", num.c, num.i, num.d);

return 0;
```

```
}
```

```
num.c = 61 를 보관하는 경우
num.c = 61, num.i = -858993603, num.d = -9.255262e+061
num.i = 541494 를 보관하는 경우
num.c = 54, num.i = 541494, num.d = 1.149050e-308
num.d = 3.141592 를 보관하는 경우
num.c = 122, num.i = -57999238, num.d = 3.141592e+000
계속하려면 아무 키나 누르십시오 . . .
```

공용체

- 예제 12.9 (ex12_9.c) (1)

```
#include <stdio.h>
```

```
struct Student {  
    int id;  
    char name[16];  
    int age;  
};
```

```
union Register {  
    char ch;  
    short int sh;  
    long int lo;  
};
```

```
void main()  
{  
    struct Student stu1;  
    union Register reg1;
```

공용체

- 예제 12.9 (ex12_9.c) (2)

```
printf("stu1의 메모리 크기 : %d \n", sizeof(stu1));  
printf("reg1의 메모리 크기 : %d \n", sizeof(reg1));
```

```
reg1.lo = 0x12345678;  
printf("reg1.ch = %8x \n", reg1.ch);  
printf("reg1.sh = %8x \n", reg1.sh);  
printf("reg1.lo = %8x \n", reg1.lo);
```

```
}
```

```
stu1의 메모리 크기 : 24  
reg1의 메모리 크기 : 4  
reg1.ch =      78  
reg1.sh =     5678  
reg1.lo = 12345678  
계속하려면 아무 키나 누르십시오 . . .
```

열거체

- 열거체(혹은 열거형) (1)

- 열거체 혹은 열거형(enumeration)이라고 하는 것은 변수가 가질 수 있는 값들을 미리 열거해 두어, 열거된 값들 외에는 변수에 대입할 수 없도록 한정해 둔 일종의 사용자 정의 자료형
- const 제한자와 비슷한 기능으로 기호 상수를 만드는 방법으로 사용할 수 있다.
- 열거형 변수에는 정해진 값만 대입될 수 있으므로 변수의 값이 실수에 의해 이상한 값이 들어가는 것을 방지할 수 있다.
- 열거형의 정의

```
enum 열거형_이름 {  
    값1, 값2, ... ;           // 열거자 : 정수 값을 나타내는 기호 상수를 나열  
};                             // 마지막에 반드시 ';'을 붙여야 한다.
```

열거체

- 열거체(혹은 열거형) (2)

- (예)

```
enum days { SUN, MON, TUE, WED, THU, FRI, SAT }; // 요일
// days형 열거형의 정의
enum colors { black, brown, red, orange, yellow, green,
blue, violet, gray, white }; // color code
// colors형 열거형의 정의
```

```
enum days d; // days형 열거형 변수 d를 선언
```

- 위에 정의된 days형 변수는 SUN, MON, TUE, ... SAT 중에서 하나의 값만을 가질 수 있다.
 - 열거자들은 정수형이며 int형으로 처리된다.
 - days 안에 들어있는 값 식별자들은 나열된 순서대로 컴파일러에 의해 0에서 시작하여 1씩 증가하는 것으로 자동으로 설정된다. 즉, SUN=0부터 1씩 증가하여 SAT=6이 된다.

열거체

- 열거체(혹은 열거형) (3)

- 열거자들의 값들의 배정은 사용자가 식별자에 값들을 배정하여 변경할 수 있다.

- (예)

enum days { SUN=1, MON, TUE, WED, THU, FRI, SAT };

- 위와 같이 하면 식별자들의 값은 SUN=1부터 1씩 증가하며 SAT=7이 된다.

열거체

- 예제 12.10 (ex12_10.c)

```
#include <stdio.h>

enum seasons { spring, summer, autumn, winter };
char *s_name[] = {"spring", "summer", "autumn", "winter"};

void main()
{
    enum seasons s;

    s = spring;
    printf("s : %d --> season : %s \n", s, s_name[s]);
    s = winter;
    printf("s : %d --> season : %s \n", s, s_name[s]);
}
```

```
s : 0 --> season : spring
s : 3 --> season : winter
계속하려면 아무 키나 누르십시오 . . .
```


열거체

- 예제 12.11 (ex12_11.c)

```
#include <stdio.h>
```

```
enum colors { black, brown, red, orange, yellow, green, blue = 16, violet, gray, white };  
typedef enum colors color_band;
```

```
void main()
```

```
{
```

```
    color_band code;  
    int value;
```

```
    code = orange;  
    printf("code = orange --> %d \n", code);  
    code = gray;  
    printf("code = gray --> %d \n", code);  
    value = red + yellow;  
    printf("value = red + yellow --> %d \n", value);
```

```
}
```

```
code = orange --> 3  
code = gray --> 18  
value = red + yellow --> 6  
계속하려면 아무 키나 누르십시오 . . .
```

Question & Answer

